

Alineación Memoria entre Copro y Sw

1. Objetivo

Actualmente el copro ORB genera los resultados con un tamaño no estándar. Se trata de adaptar estos datos para que tengan tamaños estándar y puedan ser leídos y reinterpretados por el Sw de forma más eficiente. En la mayoría de los casos el procedimiento de adaptación consistirá en hacer el padding de los datos.

2. Análisis

El código del **testbench** del bloque RS_BRIEF, nos muestra como están codificados los datos en el stream de salida. Consiste en un stream de datos de 512bits (16 datos de 32bits ó 16 uint32 ó 64B):

- score+angle => 2B
- row+col aprox. 3B
- desc aprox. 59B

```
// Imagen original 640x480 (deberían ser height 9 width 10)
HEIGHT_BIT 9 // depende del tamaño de la imagen original
WIDTH_BIT 11 // depende del tamaño de la imagen original

RS_BRIEF(cfgStream, srcPixelStream, srcFASTStream, outStream);
ap_axiu<512, 1, 1, 1> outVal = outStream.read();

int kp_num = 0;
while (outVal.last!=1)
{
    ap_uint<512> outValData = outVal.data;
    cout << "FASTscore\t" << outValData.range(6, 0) << endl; // score 7bits
    cout << "angle\t" << outValData.range(15, 7) << endl; // angle 9bits
    cout << "row_ind\t" << outValData.range(16+HEIGHT_BIT-1, 16) << endl; // row 9bits
    cout << "col_ind\t" << outValData.range(16+HEIGHT_BIT+WIDTH_BIT-1,
        16+HEIGHT_BIT) << endl; // col 11bits
    cout << "desc\t";
    for (int i = 0; i < 256/32; i++) // descriptor 8 palabras de 32bits
        cout << outValData.range(16+HEIGHT_BIT+WIDTH_BIT+(i+1)*32-1,
            16+HEIGHT_BIT+WIDTH_BIT+i*32) << " ";
    cout<<"\n-----\n";

    kp_num++;
    outVal = outStream.read();
}
cout << "total " << kp_num << " keypoints" <<endl;
}
/*
*
*          38          31          23          15          7          0
*  +-----+-----+-----+-----+-----+-----+
*  ... |   B6   |   B5   |   B4   |   B3   |   B2   |   B1   |   B0   |
*  +-----+-----+-----+-----+-----+-----+
*
*          |      col      |      row      |      angle      |      score      |
*
*/

// El dato de salida es de 512 bits (16 palabras de 32bits o 16 uint32)
```


- b) Hacer el casting en el IP para cambiar el tipo de dato => las librerías HLS no incluyen métodos de transformación del tipo "to_float()"

